جامعة بيروت العربية
BEIRUT ARAB UNIVERSITY

ICT Competition Research Project Report

DDoS Attack Detection Using ResNeXt50-32x4d with MindSpore

by

Mohamad Al Ghoush - 202300735

Abed Al Rida Nehme - 202303149

Wafik Ibrahim - 202301238

Ahmad Sharkawi - 202300122

Electrical and Computer Engineering, Faculty of Engineering

Computer Engineering Program

2026

جامعة بيروت العربية
BEIRUT ARAB UNIVERSITY

ICT Competition Research Project Report

DDoS Attack Detection Using ResNeXt50-32x4d with MindSpore

by

Mohamad Al Ghoush - 202300735

Abed Al Rida Nehme - 202303149

Wafik Ibrahim - 202301238

Ahmad Sharkawi - 202300122

Electrical and Computer Engineering, Faculty of Engineering

Computer Engineering Program

Presented to

Eng. Mustafa Bizri

2026

Abstract

This report describes a deep learning system for detecting SYN flood Distributed Denial-of-Service (DDoS) attacks from raw network traffic. Traffic is converted into grayscale images under a **0.3 s time-window** aggregation (primary model `per_second_0p3_focal_v2`), trained and evaluated on splits from `images_per_second_window_0p3` derived from the public CICDDoS2019 dataset. A ResNeXt-50 (32×4d) convolutional neural network (CNN) in Huawei MindSpore with `mindcv` performs binary classification (DDoS vs. benign), using focal loss and DDoS-focused early stopping. Reported test accuracy is about **98.1%** with macro F1 **0.937** (offline same-domain evaluation, 2026-04-03). A companion live pipeline on CPU logs window-level predictions; those lines are **not** calibrated to offline accuracy without labeled capture in deployment.

Keywords: SYN flood DDoS; packet-to-image encoding; ResNeXt; MindSpore; CICDDoS2019; intrusion detection.

Table of Contents

List of Tables

List of Figures

List of Equations

List of Abbreviations

CE cross-entropy.

CNN convolutional neural network.

DDoS Distributed Denial-of-Service.

F1 F1 score.

IDS intrusion detection system.

SYN synchronize TCP flag.

Chapter 1

Introduction

DDoS attacks, in particular synchronize TCP flag (SYN) floods, exhaust server resources by abusing the TCP three-way handshake. Signature-based and shallow learning intrusion detection system (IDS) approaches can miss novel or obfuscated attack patterns. This work follows the packet-to-image paradigm: raw packet bytes are rendered as two-dimensional images and classified by a deep CNN, enabling the model to learn spatial structure in byte layouts without hand-crafted flow statistics.

The implementation targets the Huawei ICT competition context: the full stack runs on Huawei MindSpore [1], with ResNeXt-50 [2] as the classifier and reproducible scripts for PCAP ingestion, labeling, conversion, training, and blind evaluation on CICDDoS2019 [3].

1.1 Problem Statement

Operational networks need a detector that (i) distinguishes SYN flood DDoS from benign traffic with high recall on attacks, (ii) generalizes beyond trivial rules, and (iii) can be trained and deployed with a clear, auditable pipeline. This project addresses that gap using image-based deep learning and an open dataset.

1.2 Research Objectives

- Build a reproducible pipeline from PCAP archives to labeled images and to train/validation/test splits.
- Train ResNeXt-50 (32×4d) with MindSpore, mitigating class imbalance via weighted

or focal loss where configured.

- Report standard metrics (accuracy, per-class precision/recall, macro F1 score (F1)), and large blind tests without label leakage.
- Document a practical inference path (short capture windows, CPU inference) aligned with deployment constraints.

1.3 Contributions

This work extends the ResNet-based SYN flood detection methodology described in related BAU work [4] by: (1) replacing ResNet-50 with ResNeXt-50 (32×4d) for richer feature learning via grouped convolutions; (2) implementing training, evaluation, and deployment entirely in MindSpore with `mindcv`; (3) providing end-to-end automation from CICDDoS2019 zips to checkpoints and blind evaluation; (4) supporting optional per-time-window and imbalance-aware training variants in code.

1.4 Report Organization

Chapter 2 reviews related approaches. Chapter 3 describes dataset construction, image encoding, model, and training. Chapter 4 summarizes empirical results. Chapter 5 concludes and outlines future work. The appendices cover threat model, ethics, extended related-work table, FAQ, reproducibility, optional generated figures, and future feature visualization.

1.5 Notation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ denote labeled packet images with $y_i \in \{\mathrm{DDoS}, \mathrm{normal}\}$. Training minimizes a classification loss; weighted cross-entropy (CE) with class weights w_c is

$$\mathcal{L}_{\mathrm{WCE}} = -\frac{1}{N} \sum_{i=1}^{N} w_{y_i} \log p_{y_i}(\mathbf{x}_i) \tag{1}$$

where $p_y(\mathbf{x})$ is the predicted probability for class y.

Chapter 2

Literature Review

2.1 Survey

2.1.1 Classical and feature-based DDoS detection

Traditional IDS often rely on thresholds, signatures, or statistical features over flows. These methods are interpretable but may degrade when attackers vary rates or payloads. Machine learning on engineered features improves flexibility yet still depends on feature design and domain expertise.

2.1.2 Deep learning on traffic representations

Convolutional networks applied to traffic images or matrices map raw or aggregated traffic into a grid, letting the network learn discriminative patterns. The ResNet-based SYN flood study referenced in this project [4] established a strong baseline on CICDDoS2019 using packet images and ResNet-50.

2.1.3 ResNeXt and MindSpore

ResNeXt [2] reformulates residual blocks with grouped convolutions (“cardinality”), often improving accuracy-parameter trade-offs for visual tasks. Huawei MindSpore [1] provides a graph-mode execution path suitable for training and for CPU or NPU deployment. Together, they align the method with competition and cloud tooling.

2.1.4 Comparison summary

Table 2.1 contrasts positioning of this work with common families of detectors.

Table 2.1: High-level comparison of detection approaches (qualitative).

Approach	**Strengths**	**Limitations**
Rules / thresholds	Fast, explainable	Brittle to novel attacks
ML on flow features	Adapts to data	Feature engineering burden
CNN on packet images (ResNet) [4]	Raw-byte patterns, strong CIC results	Single architecture choice
This work (ResNeXt + MindSpore)	Grouped convolutions; Huawei stack; full pipeline	Depends on labeling and windowing choices

Chapter 3

Materials and Methods

3.1 Pipeline and model

3.1.1 Dataset: CICDDoS2019

We use the public CICDDoS2019 dataset [3], which contains realistic benign and attack traffic including SYN flood scenarios. Data are split into training, validation, and test image folders (`ddos/`, `normal/`) after preprocessing.

3.1.2 PCAP labeling and packet-to-image encoding

The repository implements a lightweight labeling rule: PCAP files with more than a fixed threshold of TCP SYN packets are treated as DDoS; others as normal. Packets are converted to grayscale images by mapping raw bytes to pixel values; images are resized to 224×224. For DDoS samples, conversion may use SYN-only packets; for normal traffic, all packets may be used—matching the published methodology in the system description. Scripts include `classify_pcap_syn.py`, `pcap_to_images.py`, and batch drivers such as `convert_pcaps_to_images.sh` and `process_zips_in_batches.sh`.

3.1.3 End-to-end pipeline

Figure 3.1 summarizes the flow from raw captures to predictions.

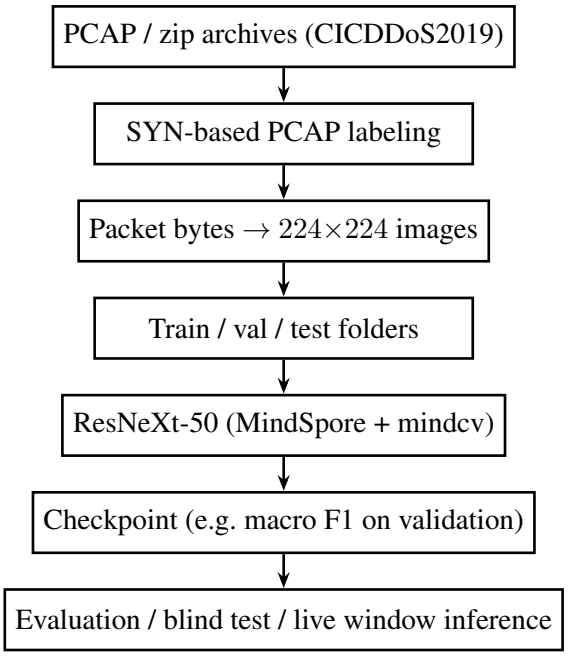


Figure 3.1: End-to-end processing and inference pipeline for SYN flood detection.

3.1.4 Model and input preprocessing

The classifier is ResNeXt-50 (32×4d), two output units (DDoS vs. normal), constructed via `mindcv`. Grayscale images are replicated to three channels. Training-time augmentation typically includes random resized crop and horizontal flip; evaluation uses resize to 256, center crop to 224, and ImageNet normalization means and standard deviations, consistent with the project's evaluation and deployment notes.

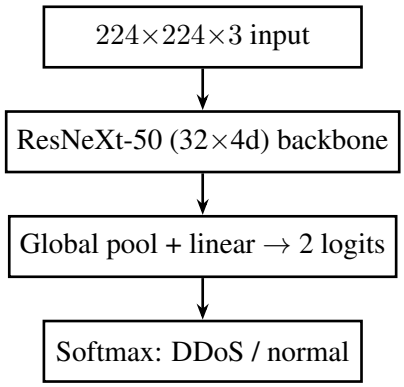


Figure 3.2: Simplified inference graph for the binary classifier (MindSpore + `mindcv`).

3.1.5 Training configuration

Training is implemented in `notebook/train_resnext.py`. The primary 0.3 s experiment is launched via `scripts/train_resnext_per_second_0p3.sh`: focal loss ($\gamma = 2.0$), minority boost 8.0, early stopping on validation DDoS F1, cosine learning rate schedule (e.g. 5×10^{-4}), batch size 32, up to 40 epochs. Output directory defaults to `model/per_second_0p3_focal_v2/`. Older runs may use weighted CE on per-packet images; checkpoints can be selected using macro F1 or class-specific F1. GPU training is supported via helper scripts; deployment has been validated with CPU inference.

Table 3.1 summarizes the hyperparameters used for the primary 0.3 s run.

Table 3.1: Primary training hyperparameters (`train_resnext_per_second_0p3.sh`).

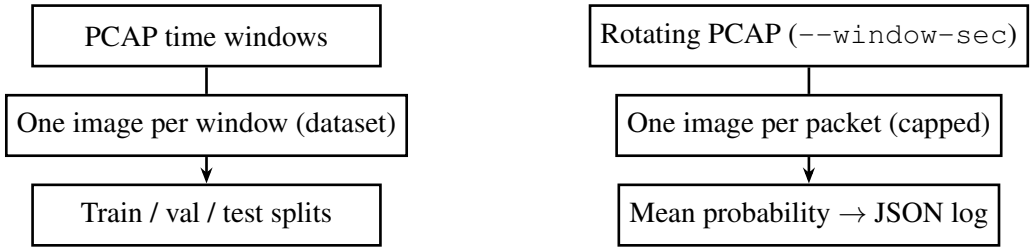
Setting	**Value**
Data root	`dataset/images_per_second_window_0p3`
Loss	Focal ($\gamma = 2.0$), minority boost 8.0
Early stopping	Validation DDoS F1 (`f1_ddos`), patience 10 epochs
Epochs (max)	40
Batch size	32
Learning rate	5×10^{-4} (cosine to 10^{-6}, 1 warmup epoch)
Optimizer class	Adam-style (as in `train_resnext.py`)
Checkpoint	`resnext50_32x4d_best.ckpt` under output dir

3.1.6 Evaluation protocol

Held-out test accuracy and F1 are reported from the training script. Additionally, `blind_test_random.py` can sample shuffled images from the test split so the model sees images only—reducing accidental order bias. Fixed random seeds support reproducibility.

3.1.7 Summary of tests performed (reproducibility)

Table 3.2 lists the evaluations that back the numbers in this report. Commands live under `scripts/`; the one-shot driver is `run_all_eval_0p3.sh`.



(a) Offline 0.3 s pipeline (training and benchmark).

(b) Live deployment (`run_pipeline.py`).

Figure 3.3: Conceptual contrast: offline window-level images vs. live per-packet images (possible distribution shift).

Table 3.2: Testing performed for the primary 0.3 s model (see also `reports/eval_results_per_second_0p3.md`).

Test	**Role**
Val / test via `eval_test_detailed.py`	Full-split metrics, confusion matrix, per-class P/R/F1 on `images_per_second_window_0p3`.
Blind sample (`blind_test_random.py`)	Shuffled images only; max 270 balanced on current test split.
Live pipeline (ECS)	Qualitative: service health, JSON lines in `detections.log`; **no** per-window ground truth, so no accuracy on production traffic.

Chapter 4

Results and Discussion

4.1 Evaluation outcomes

4.1.1 Primary results: 0.3 s time-window model

Table 4.1 summarizes **same-domain** evaluation on `dataset/images_per_second_window_0p3` using checkpoint `model/per_second_0p3_focal_v2/resnext50_32x4d_best.ckpt` (2026-04-03 evaluation). Reproduce with `bash scripts/run_all_eval_0p3.sh`; the script defaults to **GPU** (`DEVICE=CPU` for hosts without CUDA). Full confusion matrices are in `reports/eval_results_per_second_0p3.md`. Table 4.2 gives confusion counts; Table 4.3 lists precision, recall, and F1 per class.

Table 4.1: Offline metrics for the 0.3 s window model (val/test splits).

Metric	**Validation**	**Test**
Samples	1842	1845
Accuracy	98.21%	98.10%
Macro F1	0.940	0.937
Weighted F1	0.983	0.982
Balanced accuracy	0.990	0.990

Table 4.2: Confusion matrices (rows = true class, columns = predicted). Same checkpoint and date as Table 4.1.

	Validation		**Test**		
	pred. DDoS	pred. normal	pred. DDoS	pred. normal	
true DDoS	133	0	135	0	
true normal	33	1676	35	1675	

Table 4.3: Per-class metrics on validation and test (from `eval_test_detailed.py`).

	Validation			**Test**		
Class	Prec.	Recall	F1	Prec.	Recall	F1
DDoS	0.801	1.000	0.890	0.794	1.000	0.885
Normal	1.000	0.981	0.990	1.000	0.980	0.990

4.1.2 Blind test (0.3 s test split)

On the test split, class imbalance limits equal sampling: the maximum equal-per-class blind size is 270 images (135 per class). With seed 42, `scripts/blind_test_random.py` reports **98.9%** accuracy and macro F1 **0.989** (three normal images misclassified as DDoS). Table 4.4 shows the blind-test confusion matrix.

Table 4.4: Blind test ($N = 270$, seed 42, shuffled order). Rows = true class.

	pred. DDoS	**pred. normal**
true DDoS	135	0
true normal	3	132

4.1.3 Legacy baseline: per-packet images

An earlier per-packet pipeline on `dataset/images` achieved about 95.56% test accuracy and a large 3000-image blind test with 99.9% accuracy (system description). That setup is distinct from the 0.3 s window training distribution. Table 4.5 contrasts the two experimental tracks.

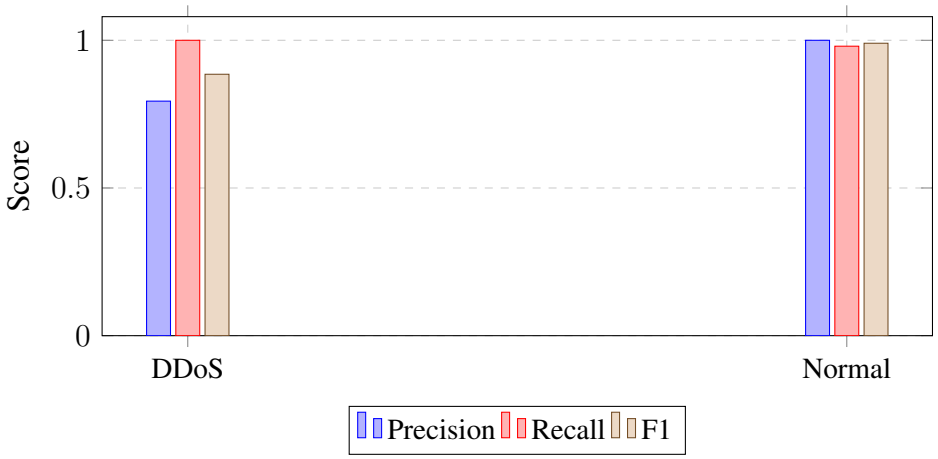


Figure 4.1: Per-class precision, recall, and F1 on the **test** split (Table 4.3).

Table 4.5: Two experimental tracks in the repository (qualitative summary).

	Per-packet images	**0.3 s window images (primary)**
Typical data path	`dataset/images`	`dataset/images_per_second_window_0p3`
Reported test acc.	∼95.6%	98.10%
Large blind test	3000 images (∼99.9% acc.)	max 270 balanced (98.9%)
Loss / selection	Weighted CE / macro F1 (typ.)	Focal, early stop on val DDoS F1

4.1.4 Deployment-oriented inference

Live `run_pipeline.py` builds **per-packet** images within each PCAP window; offline 0.3 s training uses **window-aggregated** images. The systemd unit uses `--window-sec 0.3` to align window duration with training; monitor `detections.log` and tune `--max-images-per-window` as needed. CPU inference uses the deployed checkpoint `model/per_second_0p3_focal_v2/resnext50_32x4d_best.ckpt` on the cloud host. Each log line records `pred` $\in \{\text{ddos}, \text{normal}\}$ and `p_ddos`; this is the classifier output, not independent verification of an attack.

4.1.5 Operational note: live traffic without labels

On a public interface, many windows may receive `"pred": "ddos"` even when no controlled attack was run: Internet background traffic (scans, bursts, heterogeneous protocols) can differ from CICDDoS2019, and argmax labels treat slight preference for the DDoS class as a full "DDoS" decision. High **reported** attack rates in logs therefore indicate **model**

suspicion, not confirmed incidents. Operational use should consider a confidence threshold on `p_ddos`, consecutive-window rules, or hybrid signals—left for future work.

4.1.6 Limitations

Same-domain offline scores do not guarantee the same error rate on the live path (different image construction and traffic mix). Performance also depends on how closely deployment traffic matches CICDDoS2019 labeling and windowing. Per-window and imbalance-focused runs may differ from the default image-folder training; adversarial traffic and payload encryption are out of scope for this baseline.

Chapter 5

Conclusion and Future Work

This report presented an image-based SYN flood DDoS detector using ResNeXt-50 and MindSpore, with the primary model trained on 0.3 s time-window images (`per_second_0p3_focal_v2`). On held-out test data from the same distribution, accuracy was about 98.1% with macro F1 about 0.94; blind sampling confirmed strong performance within the limits of class balance.

Future work includes validation on additional datasets and attack types; systematic comparison of time-window lengths and dual-view (all-packets vs. SYN-only) fusion; hardware acceleration beyond CPU-only deployment; stress-testing under concept drift, adversarial perturbations, and **domain adaptation** when live packet-to-image rules differ from offline windows; and **calibration** of live alerts (thresholds on `p_ddos`, temporal smoothing, or hybrid rules) to reduce false positives on real Internet traffic.

Chapter 6

Appendices

6.1 Threat model and scope

The detector is trained on **CICDDoS2019-derived** images with labels from a **SYN-centric heuristic** (PCAPs with many SYN flags treated as attack traffic). It estimates whether a batch of packet images "looks like" the DDoS class in that dataset. It does **not** by itself prove an ongoing Internet-scale attack, identify the attacker, or cover non-TCP floods, UDP floods, application-layer attacks, or encrypted traffic where byte patterns differ from the training regime.

6.2 Ethics and responsible use

Capturing live traffic may include sensitive metadata. Logs should be **access-controlled**; avoid publishing raw PCAPs from production. **Misuse**: raising false alarms can disrupt operations; automated blocking based solely on this model is discouraged without human review and tuning. **Dual-use**: the same pipeline could be profiled for evasion; document defensive intent and legal authorization for monitoring.

Symbol	Meaning
$\mathcal{D}$	Labeled image dataset
y_i	Ground-truth class index (DDoS vs. normal)
$p_{\text{ddos}}(\mathbf{x})$	Softmax probability for DDoS class
τ	Threshold on p_{ddos} for operational alerts (optional)

Table 6.1: Notation used in Section 1.5 and the appendices.

Reference	Data / setting	Model family	Notes
Bazzi et al. (BAU) [4]	CICDDoS2019, packet images	ResNet-50	Baseline for packet-image SYN detection
Sharafaldin et al. [3]	CICDDoS2019 release	n/a (dataset)	Public DDoS benchmark
Xie et al. [2]	ImageNet / vision	ResNeXt	Cardinality / grouped convolutions
Mirsky et al. [5]	Network IDS context	Autoencoder kitsune	Deep traffic anomaly framing
Vinayakumar et al. [6]	Survey	CNN/RNN/ML	Broad DL-for-ID survey
This work	CICDDoS2019, 0.3 s windows	ResNeXt-50, MindSpore	Huawei stack + live pipeline

Table 6.2: Positioning sketch (not an exhaustive literature review).

6.3 Notation reference

6.4 Extended related work (selected)

6.5 Frequently asked questions

Why do live logs show many `ddos` lines without an attack? The model performs **binary classification** on captured bytes; public Internet traffic and train/serve mismatch can yield false positives. See Section 4 (operational note).

What do `alert` and `streak` mean in JSON logs? `streak` counts consecutive windows where p_{ddos} meets `alert_min_p_ddos`; `alert` is true when the streak reaches `alert_consecutive`. Webhooks fire only when `alert` is true.

How do I reproduce the PDF figures? Run `bash scripts/run_all_paper_figures.sh` (requires matplotlib and the evaluation dataset).

6.6 Reproducibility checklist

Item	Snapshot (2026-04-03 GPU eval)
Python	3.10.12
MindSpore	2.6.0
`mindcv`	0.3.0
Git commit	`96b82d4cf152053175950a346bdd204b6e109ebf`

Table 6.3: Environment pinned for offline reproduction; see `reports/REPRODUCIBILITY.md` and `reports/pip_freeze_2026-04-03.txt`.

- **Code:** Huawei MindSpore + `mindcv`; Python 3 with project venv.
- **Checkpoint:** `model/per_second_0p3_focal_v2/resnext50_32x4d_best.ckpt`.
- **Data:** `dataset/images_per_second_window_0p3` (train/val/test).
- **Offline eval:** `bash scripts/run_all_eval_0p3.sh`.
- **Curves / ECE:** `scripts/eval_roc_pr_calibration.py` (defaults to **GPU**; `--device_targetCPU` if needed).
- **Live:** `run_pipeline.py --live` with systemd unit in `depployemnt/systemd/`.
- **Log dashboard (static HTML):** `scripts/generate_log_dashboard.py --log /path/to/detections.log`.

6.7 Optional analysis figures (generated)

ROC, PR, threshold sweep, and confusion heatmaps are written as PDFs under `research/figures/` when you run `scripts/eval_roc_pr_calibration.py` or `scripts/run_all_paper_figures.sh`. On the 2026-04-03 GPU run, `eval_roc_pr_calibration.py` reported approximate test ECE ≈ 0.074 and a max-F1 heuristic threshold ≈ 0.87 on softmax (see `reports/paper_figures_2026-04-03.log`). Include figures in slides or theses with `\includegraphics` if present.

6.8 Window-length experiments (repository scripts)

Training scripts such as `train_resnext_per_second_0p3.sh`, `train_resnext_per_second_1s.sh`, and related folders under `model/` support comparing 0.3 s vs. 1 s windows. A full sweep table is **not** fixed here because metrics depend on the machine used; run the corresponding `eval_per_second_*.sh` scripts and paste results into your defense slides.

6.9 Feature-space visualization (future work)

High-dimensional t-SNE or UMAP plots require exporting intermediate tensors from MindSpore. See `scripts/extract_features_tsne.py` for the intended hook point; not automated in this repository snapshot.

References

[1] Huawei Technologies. Mindspore: A new open source deep learning framework, 2020. URL `https://www.mindspore.cn/`. Huawei MindSpore open-source framework.

[2] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[3] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. CICDDoS2019: A dataset for distributed denial-of-service attacks. Canadian Institute for Cybersecurity dataset, 2019. URL `https://www.unb.ca/cic/datasets/ddos-dataset2019.html`.

[4] Mohammad Bazzi et al. Resnet-based detection of syn flood ddos attacks. Research report, 2022. Beirut Arab University; prior work referenced in this project.

[5] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018.

[6] R Vinayakumar, Mamoun Alazab, K.P. Soman, Prabaharan Poornachandran, Areej Al-Nemrat, and Sitalakshmi Venkatraman. Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7:41525–41552, 2019.